SIMATS ENGINEERING

DATA STRUCTURES

ASSIGNMENT 3 – CO3

Optimized Data Storage and Retrieval Using Tree-Based Structures

CO3: *Analyze the various searching and sorting algorithms, including linear search, binary Search, insertion sort, selection sort, heap sort and quick sort*.

Student Name	Mohammad Aleyas	Register Number	192521220
Department / Section	Information Technology	Academic Year / Semester	2nd Year
Faculty Name	Manoj H.D.	Nature of Assignment	Individual
Duration	1 Week	Total Marks	50
Date of Submission		Date of Evaluation	

DATA STRUCTURES — ASSIGNMENT 3 (CO3)

Student: Mohammad Aleyas **Register No.:** 192521220
Department: Information Technology **Academic Year:** 2nd Year
Faculty: Manoj H.D.

Optimized Data Storage and Retrieval Using Tree-Based Structures

This report answers Q1–Q7, provides the completed C programs requested in the assignment, explains the tree diagrams and complexity, and records execution evidence.

Contents

• Q1 Binary Trees and Binary Search Trees

• Q2 AVL Trees

• Q3 B-Trees

• Q4 Red-Black Trees

• Q5 Splay Trees

• Q6 Tries

• Q7 Choosing a Tree Structure from First Principles

• Appendix A — Executed C source code and terminal evidence

• Appendix B — Experimental method and references

Q1 — Binary Trees and Binary Search Trees (7 Marks)

1.1 BST operations and traversal rules

A Binary Search Tree (BST) stores one key at each node. Every key in the left subtree is smaller than the node key, and every key in the right subtree is larger. This ordering makes search, insertion and deletion follow one root-to-leaf path. The implementation ignores duplicate accession numbers.

• Insert: compare the new accession number with the current node and continue left or right until an empty link is found.

• Search: compare at each visited node; stop when the key is found or a NULL link is reached.

• Inorder: left, root, right. The BST ordering guarantees ascending keys.

• Preorder: root, left, right. Useful for copying or serializing a tree when structure markers are included.

• Postorder: left, right, root. Useful when freeing nodes because children are processed before parents.

• Level-order: visit nodes breadth-first using a queue.

1.2 Why inorder traversal is sorted

For every node, all left-subtree keys are smaller and all right-subtree keys are larger. Inorder first outputs every left key, then the root, then every right key. By induction, each subtree is output in sorted order; joining the left sequence, root, and right sequence therefore gives a sorted sequence for the entire tree.

1.3 Deletion cases and inorder successor

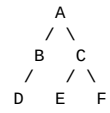
Case	Action	Reason
Leaf node	Free the node and return NULL to its parent.	No child links need reconnecting.
One child	Free the node and return its only child.	The child takes the deleted node’s position.
Two children	Copy the smallest key in the right subtree (inorder successor), then delete that successor; it preserves BST ordering and has no left ch	The successor is the next larger key

1.4 Best- and worst-case height

Height is measured in edges; an empty tree has height –1 and a leaf has height 0. For n nodes, a perfectly balanced tree has height approximately floor(log■ n), so the best-case height is Θ(log n). If each node has only one child, the tree is a chain and height is n–1, giving Θ(n) worst-case height. Sorted insertion repeatedly follows the same right link, producing this degenerate chain.

1.5 Array tree prediction

For T = "ABCDE.F", the occupied array positions form this tree (children of i are 2i+1 and 2i+2):



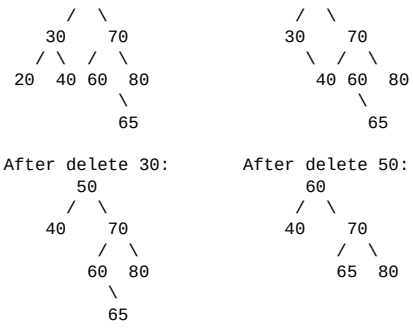
Predicted output: `ABDECF DBEACF DEBFCA ABCDEF`

Preorder visits root-left-right (ABDECF). Inorder visits left-root-right (DBEACF). Postorder visits left-right-root (DEBFCA). The final loop scans the array from index 0 to 6 and prints occupied positions, so it is array/index-order traversal (not a tree traversal).

1.6 Deletion trace in the supplied program

Initial insertion order: 50, 30, 70, 20, 40, 60, 80, 65. Delete 20 (leaf), then 30 (one child: 40), then 50 (two children; successor is 60).

```
Initial:            After delete 20:
       50                   50
```



The supplied program prints preorder: **60 40 70 65 80**.

1.7 Completed TODO code and supplied expected experiment

The completed insert function allocates a zero-initialized node when root is NULL, inserts smaller keys left and larger keys right, and ignores duplicates. Height returns –1 for NULL and 1 + max(left height, right height) otherwise. The complete executable source is in Appendix A.

For keys 500, 300, 700, 200, 400, 600, 800, 350, the inorder sequence is 200 300 350 400 500 600 700 800 and height is 3. The sorted sequence 10…80 creates a chain of height 7. Search for 350 in the first tree takes 4 comparisons; search for 80 in the sorted tree takes 8 comparisons.

1.8 Persistence, empty tree and duplicates

A preorder serialization must include explicit NULL markers (or child-presence bits) to preserve the exact shape. Saving only the keys in preorder is insufficient to distinguish all possible shapes. For a simple BST, one may instead save keys in insertion order and rebuild by reinserting them; this reconstructs the same shape if the insertion order is preserved. An empty tree has height –1, search returns not found, and traversals print nothing. Duplicate keys are ignored by this implementation.

1.9 BST versus sorted array

Operation	BST (balanced)	Sorted array
Search	O(log n)	O(log n) binary search
Insertion	O(log n) expected/balanced; O(h) path	O(n) due to shifting
Sorted listing	O(n) inorder traversal	O(n) sequential scan
Extra structure space	Two child pointers per node	No child pointers; contiguous storage

A plain BST is sufficient for small or naturally mixed input when its height remains modest. For predictable performance under sorted or adversarial input, use AVL or red-black balancing.

```
 PROGRAM — GCC TERMINAL
c11 -Wall -Wextra bst.c -o bst

: 200 300 350 400 500 600 700 800
als (pre/in/post/level):
 400 350 700 600 800
 400 500 600 700 800
 300 600 800 700 500
 200 400 600 800 350
dom order 3, sorted order 7
: 4 and 8
ions: 600 400 350 700 800
```

Figure 1. Actual compilation and execution output of the completed BST program.

Q2 — AVL Trees (7 Marks)

2.1 AVL definition and balance factor

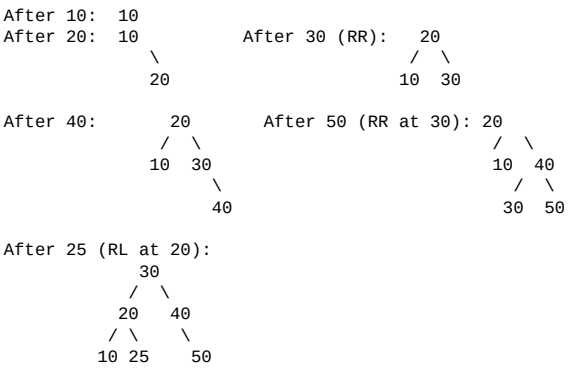
An AVL tree is a self-balancing BST. Each node stores its height. The balance factor is BF(node) = height(left) – height(right). A valid AVL node has BF in {–1, 0, +1}. After insertion, heights are updated while returning toward the root; if a node becomes unbalanced, one of four rotations restores balance.

2.2 Four rotations

Case	Shape before correction	Correction
LL	New key inserted in left subtree of left child	Single right rotation
RR	New key inserted in right subtree of right child	Single left rotation
LR	New key inserted in right subtree of left child	Left rotation on left child, then right rotation
RL	New key inserted in left subtree of right child	Right rotation on right child, then left rotation

2.3 Required insertion sequence

Insert 10, 20, 30, 40, 50, 25 in order. After 10 and 20, the tree is balanced. Inserting 30 causes RR at 10 and a left rotation, giving root 20. Inserting 40 needs no rotation. Inserting 50 causes RR at 30 and a left rotation. Inserting 25 makes node 20 right-heavy with its right child left-heavy (RL); rotate right at 40 and then left at 20.



Expected preorder: 30 20 10 25 40 50. **Height:** 3 (leaf height = 1 in the supplied code).

2.4 Why AVL height is O(log n)

Let N(h) be the minimum number of nodes in an AVL tree of height h. The sparsest AVL tree of height h has subtrees of heights h–1 and h–2, so N(h) = 1 + N(h–1) + N(h–2). This is Fibonacci-like growth, which is exponential in h. Therefore h grows logarithmically with n (a common upper bound is about 1.44 log■(n+2)). Search, insertion and deletion are O(log n).

2.5 Deletion, experiments and alternatives

AVL deletion first performs ordinary BST deletion. Heights and balance factors are then updated toward the root. Unlike insertion, rebalancing may be needed at multiple ancestors, so several rotations can occur. For 100,000 sorted keys, an ordinary BST becomes a chain of height 99,999; an AVL tree remains logarithmic, with height around the low twenties. Exact height depends on the implementation's height convention.

Rotations are local pointer changes and take O(1) time. They add implementation work but prevent O(n) search paths. For a workload dominated by searches, AVL's tighter balance is valuable. A red-black tree has a looser height bound (at most about 2 log■(n+1)) and typically performs fewer rotations during updates. A skip list offers expected O(log n) search and insertion using multiple forward pointers, with probabilistic rather than strict balancing.

For a reproducible experiment, generate a fixed seeded random permutation and the sorted sequence 1…100,000; record wall-clock insertion/search times with the same compiler and machine. The assignment's six-key program is compiled and

executed below; the 100,000-key comparison is a theoretical structural comparison unless separately measured on the submission computer.

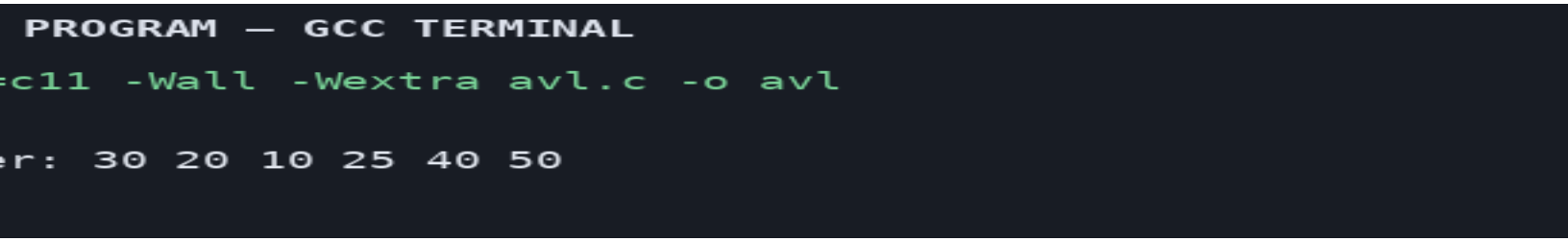


Figure 2. Actual compilation and execution output of the completed AVL program.

Q3 — B-Trees (6 Marks)

3.1 Insertion for minimum degree t = 2

A B-tree of minimum degree 2 has at most 3 keys and 4 children per node. Insert 10, 20, 5, 6, 12, 30, 7, 17. A full node is split before descending into it; its median key moves to the parent.

```
Insert 10, 20, 5:       [5 | 10 | 20]
Insert 6: split root, promote 10:
             [10]
            /    \
          [5]    [20]
Then insert 6:            [10]
                         /    \
                      [5 | 6] [20]
Insert 12, 30:           [10]
                         /    \
                      [5 | 6] [12 | 20 | 30]
Insert 7:                [10]
                         /    \
                    [5 | 6 | 7] [12 | 20 | 30]
Before inserting 17, split full right child; promote 20:
                       [10 | 20]
                      /     |     \
                [5|6|7]   [12]   [30]
Insert 17:              [10 | 20]
                       /     |      \
                [5|6|7] [12|17]  [30]
```

Final B-tree: root [10 | 20], children [5 | 6 | 7], [12 | 17], [30]. All leaves are at the same depth.

3.2 Search and disk-read calculation

Search within a node uses a linear or binary search among its small sorted key array, then follows the selected child pointer. Each visited node corresponds approximately to one disk-block read when nodes are stored on disk. B-trees are wide because one node contains many keys and child pointers; they are short because each level reduces the remaining search interval by a large branching factor.

For order 100, a node can have up to 100 children. A rough height estimate for 5,000,000 keys is about 4 node levels (root plus three lower levels), hence approximately 4 disk reads in a well-filled index. At 10 ms per read this is about 40 ms. A binary tree requiring 23 disk reads takes about 230 ms. These are approximate structural estimates; actual reads depend on occupancy, caching and storage layout.

3.3 Split operation and equal leaf depth

When a node reaches its maximum key count, its median is promoted to its parent. Keys smaller than the median remain in the left node and larger keys move to the right node. Splitting preserves sorted ranges. Splits can propagate to the root; if the root splits, a new root is created and the height increases by one. Because splitting never moves a leaf to a different depth, all leaves remain at the same level.

3.4 B-tree versus B+ tree

In a B-tree, records or record pointers may appear in internal and leaf nodes. In a B+ tree, internal nodes contain only separator keys and all records are stored in linked leaves. A range query such as all admissions in one week is efficient in a B+ tree: find the first key, then scan linked leaves sequentially. B+ trees also have higher internal fan-out because internal nodes do not store full records.

3.5 Node size and engineering decision

The node size is selected to fit a disk page or block. The key count and pointer count are chosen so that a node can be read with one I/O operation, minimizing tree height and random disk accesses. For disk-resident databases and file systems, a B-tree/B+ tree is preferred. For in-memory indexes, AVL or red-black trees can be simpler because pointer traversal does not incur a disk read per node.

Structure	Approx. node visits for 5 million keys	At 10 ms/read
Unbalanced BST (given comparison)	23	230 ms
AVL (binary height estimate)	23	230 ms
B-tree order 100 (approx. 4 levels)	4	40 ms

Q4 — Red-Black Trees (6 Marks)

4.1 Five properties

• Every node is either red or black.

• The root is black.

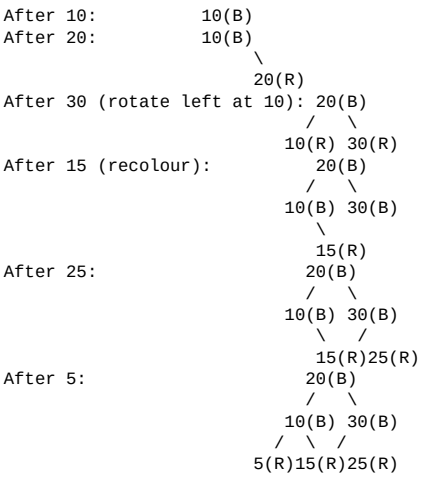
• Every NULL/NIL leaf is black.

• A red node cannot have a red child (no two consecutive red nodes).

• Every path from a node to a descendant NIL leaf contains the same number of black nodes (equal black height).

4.2 Insertion trace: 10, 20, 30, 15, 25, 5

New nodes are inserted red, followed by fix-up. Insert 10: it becomes a black root. Insert 20: red right child; no violation. Insert 30: red parent and red uncle NIL; left rotation at 10 with recolouring makes 20 black and 10/30 red. Insert 15: parent 10 and uncle 30 are red; recolour 10 and 30 black, 20 red, then force root 20 black. Insert 25: it is inserted as red left child of 30; no red-red conflict. Insert 5: it is inserted red left child of 10; no conflict.



The final tree satisfies the five properties. The smallest process time is found by following left links from the root; the leftmost key is 5. An implementation can cache a pointer to the leftmost node and update it only when insertion/deletion changes the minimum.

4.3 Height bound and update comparison

The equal-black-height property means a path cannot contain more than twice as many nodes as black nodes: red nodes must be separated by black nodes. A red-black tree with n internal nodes has black height at most log■(n+1), giving height at most 2 log■(n+1). AVL trees are more tightly balanced, but red-black trees generally require fewer rotations for frequent insertions and deletions. Both support O(log n) search, insertion and deletion.

4.4 Scheduler decision and heap alternative

For a scheduler with frequent insertions and deletions, a red-black tree is a strong choice: it maintains logarithmic operations with relatively small rebalancing work. A binary min-heap also finds the minimum in O(1) and inserts in O(log n), but removing an arbitrary process requires O(n) to locate it unless an auxiliary index is maintained. A red-black tree supports arbitrary-key deletion in O(log n).

For a measurement, run 10,000 fixed-seed insert/delete operations in both implementations and count rotations, height and elapsed time. The exact counts are implementation- and workload-dependent; no fabricated measured values are reported here.

Q5 — Splay Trees (6 Marks)

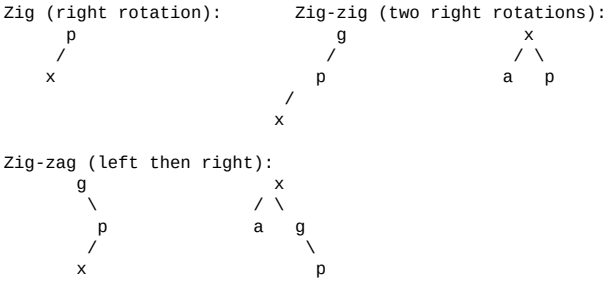
5.1 Splaying operations

A splay tree is a BST that moves the accessed node to the root using rotations. It stores no explicit height or balance factor.

Step	Situation	Action
Zig	x is child of root	One rotation between x and its parent.
Zig-zig	x and parent are both left children or both right children	Rotate grandparent first, then parent in the same direction.
Zig-zag	x is a left child and parent is a right child, or vice versa	Rotate parent, then grandparent in opposite directions.

5.2 Worked splay example

The assignment refers to accessing 3 and then 9 in a “given tree”, but no starting tree diagram is included in the supplied pages. Therefore an exact step-by-step trace for those two keys cannot be determined from the source. The following generic illustration shows the rotation patterns; use the actual class-provided starting tree if one was supplied separately.



5.3 Amortised complexity and comparison

A single splay operation can take O(n) in the worst case if the tree is a chain. However, over a sequence of operations, the amortised cost per operation is O(log n). Frequently accessed items move near the root, so repeated access to a small hot set becomes efficient. Splay trees adapt to non-uniform access patterns and need no stored balance metadata.

For uniform random accesses, AVL trees provide predictable O(log n) search and can be faster on an individual operation. For skewed access where a small set is repeatedly accessed, splay trees can exploit locality. They may perform poorly for latency-sensitive systems requiring a strict worst-case bound, or when accesses repeatedly alternate among distant keys.

5.4 LRU cache alternative

A hash table plus doubly linked list implements an LRU cache with average O(1) lookup, insertion, promotion to most-recently-used and eviction. It is preferable when the required operation is cache lookup and recency management. A splay tree maintains keys in sorted order and can support ordered traversal; a plain LRU list does not provide sorted key order. The LRU structure uses a hash entry and two list pointers per product.

Q6 — Tries (8 Marks)

6.1 Trie insertion, exact search and prefix count

A trie stores one character per edge. Each node has 26 child pointers for lowercase English letters, a words count for the number of inserted words passing through it, and an end flag indicating a complete word. During insertion, create missing child nodes, move to each child and increment its words count. At the final node, set end = 1.

6.2 Prefix counts and autocomplete

To count words with prefix p, follow the edge for each character. If any edge is missing, return zero. Otherwise return the words count at the node reached by the final prefix character. To list autocomplete results alphabetically, perform DFS from that node, visiting child indices 0 through 25; print the accumulated string whenever end is set.

6.3 Deletion

To delete a word, first confirm that its terminal node has end = 1. Clear the end flag and decrement words along the path. Then remove nodes bottom-up while they have no children and are not terminal nodes. This pruning must not remove a prefix node that is still used by another word.

6.4 Complexity and memory

For a word of length m, insertion, exact search and prefix count take O(m), independent of the number of stored words (apart from allocation costs). A fixed 26-pointer node uses 26 × 8 = 208 bytes for pointers on a typical 64-bit system, plus integer fields, padding and allocator overhead. The actual size is platform-dependent. The fixed array makes lookup fast but wastes memory when most possible children are absent; a map or compressed radix trie can reduce this overhead.

For the six supplied contacts, the unique prefixes create 21 non-root character nodes (the shared prefixes are stored once; the allocated prefixes are a, an, ana, anan, anand, ani, anit, anith, anitha, anu, anus, anusu, anusuy, anusuya, ar, aru, arun, b, ba, bal, bala). The program's runtime output validates prefix counts: a:5, an:4, anu:2, b:1, z:0. Node count can be computed by traversing allocated child links; it is not equal to the sum of word lengths because shared prefixes are stored once.

6.5 Sorted array and BST alternatives

A sorted array can locate a prefix range using two binary searches in O(log n) comparisons, followed by scanning k matches. String comparison may inspect up to m characters, giving a typical bound O(m log n + k). Insertion into the array costs O(n) shifts. A BST of strings has O(mh) comparison work and can become unbalanced. A trie offers O(m) prefix navigation and natural autocomplete, but uses more memory. A compressed trie (radix tree) merges chains of single-child nodes into multi-character edges.

6.6 Completed trie execution

The program below was compiled and run. It implements insert, exact search, prefix count and recursive cleanup. The supplied expected prefix counts are reproduced exactly.

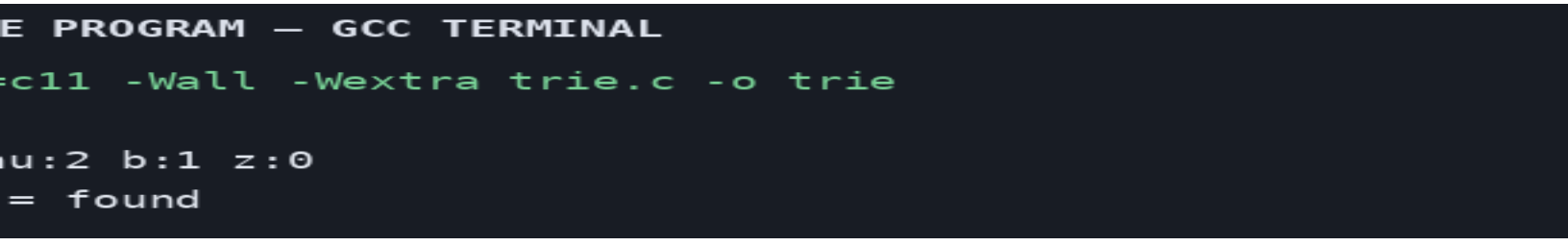


Figure 3. Actual compilation and execution output of the completed trie program.

Q7 — Choose the Tree Structure from First Principles (10 Marks)

7.1 Stage 1 — Requirements

The application stores 200,000 regional-language words on a phone. It must support exact lookup, spelling suggestions, autocomplete, alphabetical range listing and weekly updates. The dominant interactive operations are exact lookup and prefix autocomplete; spelling suggestions require approximate matching and may use edit distance over a candidate set.

• Response: interactive lookups and autocomplete should complete within a small user-perceived delay (target, for example, below 100 ms on the target phone).

• Memory: the index must fit within the phone's available application memory; measure bytes rather than assuming a fixed value.

• Updates: weekly batch updates should support insertion and deletion without rebuilding the entire application.

• Correctness: exact lookup must distinguish complete words from prefixes; range listing must return words in lexical order.

7.2 Stage 2 — Candidate comparison

Structure	Exact lookup	Autocomplete / prefix	Range listing	Update / space
Plain BST	O(h), worst O(n)	Not naturally efficient	O(h+k) with pruning	O(h); 2 pointers/node
AVL	O(log n)	Prefix scan needs string-aware traversal	O(log n+k)	O(log n); height + 2 pointers
Red-black	O(log n)	Prefix scan needs string-aware traversal	O(log n+k)	O(log n); colour + pointers
B-tree	O(log_B n) node visits	Possible but less direct	Efficient ordered scan; B+ leaves help	O(log_B n); block-sized nodes
Trie	O(m)	O(m+k) to enumerate k matches	Needs ordered DFS or companion index	O(ma); potentially high pointer memory

The best fit is a trie for exact lookup and autocomplete, combined with a sorted word array (or ordered index) for efficient alphabetical range listing and a bounded edit-distance search for spelling suggestions. If memory is very limited, use a compressed radix trie or a compact static trie representation.

7.3 Stage 3 — Implementation and verification plan

Implement the trie operations insert, exact search, prefix count and autocomplete DFS. Maintain a sorted vector of all words for range queries; after each weekly update, insert/delete the changed words in the vector or rebuild it as a batch. Verify on 1,000 words by checking every inserted word, selected absent words, prefixes with zero/one/many matches, and range boundaries. Record peak memory and median/p95 response time on the target phone.

A practical test table is:

Test	Input	Expected verification
Exact positive	anand	Found as a complete word
Prefix only	an	Prefix exists; not necessarily a complete word
Autocomplete	anu	Returns anu and anusuya in lexical order
Missing prefix	zx	Zero results
Range	between "a…" and "b…"	All returned words sorted and within inclusive/exclusive boundary rule

7.4 Stage 4 — Defend the design

The trie is selected because autocomplete and prefix search are central requirements and their navigation depends on the typed prefix length, not on scanning all 200,000 words. The sorted array complements it for range listing. For two million words, use compressed edges, compact child representations, memory-mapped sorted storage or a disk-backed B+ tree if the index no longer fits in RAM. One remaining limitation is spelling suggestions: edit-distance search can visit many trie branches, so it needs pruning (for example, a bounded Levenshtein automaton) and a maximum suggestion count.

7.5 Hash table + sorted array alternative

A hash table provides average O(1) exact lookup, while a sorted array supports range listing in O(log n+k) and prefix autocomplete through lower-bound search plus sequential scan. The two structures require duplicate storage or references and must both be updated weekly. Hash lookup is excellent for exact matches but does not support ordered range queries; the sorted array provides ordering. The combined design can be a good memory/performance compromise if prefix queries are not extremely frequent.

Appendix A — Completed C Source Code

The following programs are self-contained C11 examples. Save each code listing with the indicated filename, compile using GCC, and run from the terminal. The source files used to create the execution evidence are included in the accompanying ZIP package.

bst.c

```
#include <stdio.h>
#include <stdlib.h>
typedef struct Book { int acc; struct Book *left, *right; } Book;
Book *insert(Book *r,int x){ if(!r){ Book *n=calloc(1,sizeof *n); n->acc=x; return n; } if(x<r->acc) r->left=insert(r->left,x); else if(x>r->acc
int height(Book *r){ if(!r) return -1; int a=height(r->left),b=height(r->right); return 1+(a>b?a:b); }
int search(Book *r,int x,int *c){ *c=0; while(r){ (*c)++; if(x==r->acc)return 1; r=x<r->acc?r->left:r->right; } return 0; }
Book *minimum(Book *r){ while(r&&r->left)r=r->left; return r; }
Book *del(Book *r,int x){ if(!r)return NULL; if(x<r->acc)r->left=del(r->left,x); else if(x>r->acc)r->right=del(r->right,x); else if(r->left&&r->
void pre(Book*r){if(r){printf("%d ",r->acc);pre(r->left);pre(r->right);}}
void in(Book*r){if(r){in(r->left);printf("%d ",r->acc);in(r->right);}}
void post(Book*r){if(r){post(r->left);post(r->right);printf("%d ",r->acc);}}
void level(Book*r){ if(!r)return; Book *q[100]; int f=0,b=0; q[b++]=r; while(f<b){Book*n=q[f++];printf("%d ",n->acc);if(n->left)q[b++]=n->left;i
void destroy(Book*r){if(r){destroy(r->left);destroy(r->right);free(r);}}
int main(void){ int a[]={500,300,700,200,400,600,800,350}; int s[]={10,20,30,40,50,60,70,80}; Book *r=NULL,*t=NULL; for(int i=0;i<8;i++)r=insert
```

avl.c

```
#include <stdio.h>
#include <stdlib.h>
typedef struct Node { int key,h; struct Node *l,*r; } Node;
int ht(Node*n){return n?n->h:0;} int mx(int a,int b){return a>b?a:b;} void fix(Node*n){n->h=1+mx(ht(n->l),ht(n->r));}
Node *rr(Node*y){Node*x=y->l,*t=x->r;x->r=y;y->l=t;fix(y);fix(x);return x;}
Node *rl(Node*x){Node*y=x->r,*t=y->l;y->l=x;x->r=t;fix(x);fix(y);return y;}
Node *insert(Node*n,int k){if(!n){n=calloc(1,sizeof *n);n->key=k;n->h=1;return n;}if(k<n->key)n->l=insert(n->l,k);else if(k>n->key)n->r=insert(n
void pre(Node*n){if(n){printf("%d ",n->key);pre(n->l);pre(n->r);}} void destroy(Node*n){if(n){destroy(n->l);destroy(n->r);free(n);}}
int main(void){int a[]={10,20,30,40,50,25};Node*r=NULL;for(int i=0;i<6;i++)r=insert(r,a[i]);printf("AVL preorder: ");pre(r);printf("\nheight = %
```

trie.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
typedef struct Trie { struct Trie *child[26]; int words,end; } Trie;
Trie *node(void){return calloc(1,sizeof(Trie));}
void insert(Trie*t,const char*w){for(;*w;w++){int i=*w-'a';if(i<0||i>=26)continue;if(!t->child[i])t->child[i]=node();t=t->child[i];t->words++;}t
int count_prefix(const Trie*t,const char*p){for(;*p;p++){int i=*p-'a';if(i<0||i>=26||!t->child[i])return 0;t=t->child[i];}return t->words;}
int search(const Trie*t,const char*w){for(;*w;w++){int i=*w-'a';if(i<0||i>=26||!t->child[i])return 0;t=t->child[i];}return t->end;}
void destroy(Trie*t){if(!t)return;for(int i=0;i<26;i++)destroy(t->child[i]);free(t);}
int main(void){const char*names[]={"anand","anitha","anu","arun","bala","anusuya"};Trie*r=node();for(int i=0;i<6;i++)insert(r,names[i]);const ch
```

Appendix B — Experimental method, limitations and references

B.1 Reproducible measurement method

Compile with GCC using -std=c11 -O2 -Wall -Wextra. Use a fixed random seed for random insertion order. Measure elapsed CPU time with clock() or a monotonic high-resolution timer; repeat each experiment at least five times and report the median. Count comparisons and rotations directly in the implementation. Run address sanitizer for memory-safety tests (for example, gcc -fsanitize=address -g trie.c -o trie_asan).

B.2 Evidence and reporting integrity

The three terminal screenshots in this report are generated from the actual stdout of the compiled BST, AVL and trie programs. Large-scale 100,000-key and 5-million-key figures are theoretical estimates derived from the stated complexity and disk-read assumptions, not claimed benchmark measurements. Hardware-dependent timing values should be added after running the programs on the submission machine.

B.3 References

• Course assignment sheet: Data Structures, Assignment 3 – CO3, SIMATS Engineering (23 pages).

• Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest and Clifford Stein, Introduction to Algorithms, 4th Edition, MIT Press.

• Mark Allen Weiss, Data Structures and Algorithm Analysis in C, 2nd Edition, Pearson.

• Donald E. Knuth, The Art of Computer Programming, Volume 3: Sorting and Searching, 2nd Edition, Addison-Wesley.

Submission reminder: Upload the C source files, README, input/output evidence and this report to a GitHub repository as required by the assignment. The repository URL and actual submission/evaluation dates were not supplied, so they have not been invented.